

TECNOLOGIA E ANÁLISE EM DESENVOLVIMENTO DE SISTEMAS

Disciplina: Algoritmo
Prof. Aluisio Igor

SEGUNDA LISTA DE EXERCÍCIOS

1. (Fácil) Explique como pode ser feita a análise do tempo de execução de um algoritmo
2. (Fácil) A análise de algoritmos mede o tempo de execução de um algoritmo em função de quê?
3. (Fácil) As classes de comportamento assintótico são usadas para a análise do comportamento do tempo de execução de um algoritmo. Quais são as principais classes de comportamento assintótico existentes?
4. (Fácil) O que é o pior caso, caso médio e melhor caso de um algoritmo?
5. (Normal) Suponha um algoritmo A e um algoritmo B com funções de complexidade de tempo $a(n) = n^2 \cdot n + 549$ e $b(n) = 49n + 49$, respectivamente. Determine quais são os valores de n pertencentes ao conjunto dos números naturais para os quais A leva menos tempo para executar do que B. Sugestão: Faça um algoritmo para encontrar esses valores
6. (Normal) Considere que você tenha um problema para resolver e duas opções de algoritmos. O primeiro algoritmo é quadrático tanto no pior caso quanto no melhor caso. Já o segundo algoritmo, é linear no melhor caso e cúbico no pior caso. Considerando que o melhor caso ocorre 90% das vezes que você executa o programa enquanto o pior caso ocorre apenas 10% das vezes, qual algoritmo você escolheria? Justifique a sua resposta em função do tamanho da entrada
7. (Normal) Defina matematicamente a complexidade do melhor caso, pior caso e o caso médio do algoritmo abaixo. Explique qual o objetivo desse algoritmo.

```
1
2 void bubble_sort(int *a)
3 {
4     int i, j, tmp;
5
```

```

6   for (i = 0; i < MAX; i++)
7   {
8       for (j = i+1; j < MAX; j++)
9       {
10          if (a[j] < a[i])
11          {
12              tmp = a[i];
13              a[i] = a[j];
14              a[j] = tmp;
15          }
16      }
17  }
18  }

```

8. (Normal) Defina matematicamente a complexidade do melhor caso, pior caso e o caso médio do algoritmo abaixo. Explique qual o objetivo desse algoritmo.
-

```

1
2  int main()
3  {
4      int n, maior = 0, menor = 0, soma = 0;
5      cout << "Introduza quantos numeros quiser e se desejar sair insira '0'";
6      do{
7          cout <<("Introduza um numero: ");
8          cin >>(n);
9          soma = soma + n;
10         if (n > maior)
11             maior = n;
12         if (n < menor && n != 0)
13             menor = n;
14     } while (n != 0);
15     cout << "Menor valor: %dnMaior valor: %dnSoma: %dn", menor, maior, soma);
16
17 }

```

9. (Normal) Defina matematicamente a complexidade do melhor caso, pior caso e o caso médio do algoritmo abaixo. Explique qual o objetivo desse algoritmo.
-

```

1
2  void MultiplyWithoutAMP() {
3
4      int aMatrix[3][2] = {{1, 4}, {2, 5}, {3, 6}};
5      int bMatrix[2][3] = {{7, 8, 9}, {10, 11, 12}};
6      int product[3][3] = {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}};
7
8      for (int row = 0; row < 3; row++) {
9          for (int col = 0; col < 3; col++) {
10             for (int inner = 0; inner < 2; inner++) {
11                 product[row][col] += aMatrix[row][inner] * bMatrix[inner][col];
12             }
13             std::cout << product[row][col] << " ";
14         }
15         std::cout << "\n";
16     }
17 }

```

10. (**Diffícil**) 8-puzzle é um jogo de tabuleiro 3x3 com peças enumeradas de 1 a 8 e um espaço vago. O objetivo é misturar as peças e posteriormente voltar ao estado original, com as peças ordenadas de maneira crescente por linha. A única operação permitida é deslizar peças adjacentes ao espaço vago. Mostre na tela o passo a passo para resolução do jogo. Obs: Não pode usar recursividade.

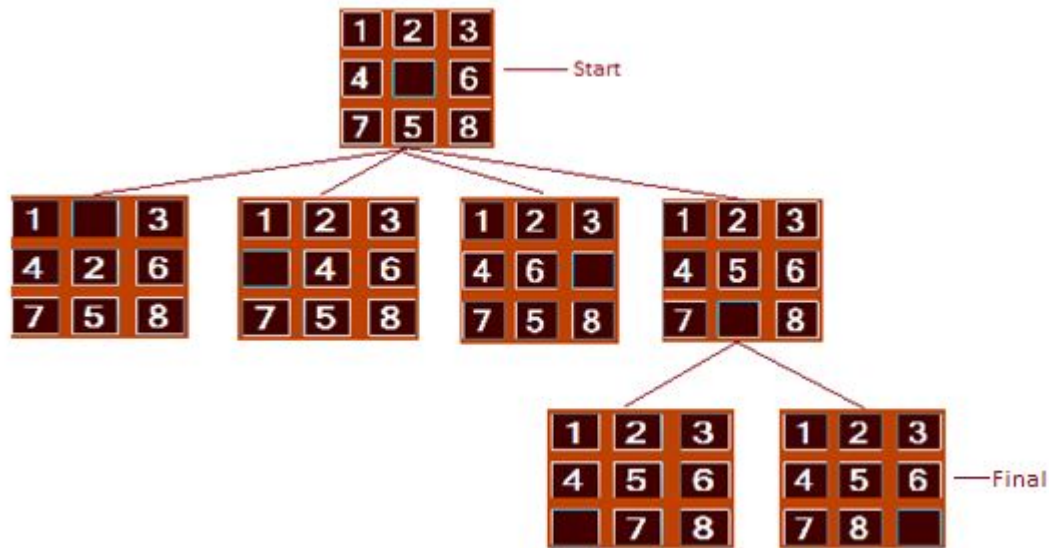


Figura 1: Passos da resolução